

Task1 : Unstructured Data Pipeline & Remote MCP Server

Task2 : Data Preprocessing as Claude Skills

系統需求 → 技術選擇 → 架構設計 → 開發過程 → 測試驗證 → 上線與使用

GitHub : <https://github.com/j84077200345-dotcom/Enterprise-Knowledge-MCP-Server>

部署公開網址 : <https://enterprise-knowledge-mcp-server.zeabur.app>

前言：兩份作業的關係

本報告涵蓋兩份彼此相關的作業，共用同一套程式庫 (Repo)：

- **作業一 (Enterprise Knowledge MCP Server)** 為一套 Production-Ready 的企業文件知識庫系統：讀取非結構化文件 (DOCX/PDF/PPTX)，經解析、清洗、具中繼資料的切塊、混合檢索索引後，封裝為 Remote MCP Server，供 Claude Desktop 等 Agent 查詢。
- **作業二 (document-preprocessing Claude Skill)** 將作業一的「文件前處理管線」(解析 → 清洗 → 切塊) 抽出、封裝為一個可重用的 Claude Skill，讓 Claude Code 或任何 Agent 都能在不啟動 HTTP / MCP Server 的情況下，對單一文件做前處理並產出結構化 JSON / Markdown。

兩者的核心價值：作業一交付「可查詢的知識庫服務」，作業二交付「可重用的前處理能力」。
作業二是作業一生產程式碼 (`app/services` 下的 `parser / cleaner / chunker`) 的薄封裝——
同一份邏輯、兩種交付形態，避免重複實作。

以下先完整說明作業一，再說明作業二。每份皆依「系統需求 → 技術選擇 → 架構設計 → 開發過程 → 測試驗證 → 上線 / 使用方式」六節撰寫。

作業一 Enterprise Knowledge MCP Server

一、系統需求

目標為建立一套 Production-Ready 的企業文件知識庫系統，需具備以下能力：

1.1 功能需求

- **文件輸入**：支援 DOCX / PDF / PPTX 三種格式；內容可能含多頁、Header/Footer、表格、圖片、Caption、投影片結構。
- **文件處理**：以 Docling 為主要 Parser（避免以簡單 PDF OCR 取代）；清洗重複頁首頁尾、空白與無意義段落，但保留文件結構與頁碼 / 投影片頁碼。
- **切塊**：Chunk 必須保留完整 Metadata，禁止固定字數切割，須沿語意邊界（Page / Section / Table / Figure）切分。
- **檢索**：採 Hybrid Search（Dense Embedding + BM25），不得僅以 SQLite FTS 作為最終方案。
- **MCP**：以 FastMCP 封裝為 Remote MCP Server，提供 search_documents / get_document / get_chunk / list_documents 四項工具，以及 documents://all、documents://{document_id} 兩項資源。

1.2 每個 Chunk 必須保存的 Metadata Schema

欄位	型別	說明
document_id	str	文件唯一 id
filename	str	原始檔名
page_number	int null	PDF/DOCX 頁碼（1-based）
slide_number	int null	PPTX 投影片編號（1-based）
chunk_id	str	唯一且可重現： <document_id>-<流水號>
chunk_type	str	text / table / figure
content	str	Chunk 內容（表格為 Markdown）
section_title	str null	所屬標題（額外提供的檢索上下文）

1.3 非功能需求 (題目明確要求)

- 需部署至公開網址 (Zeabur / Railway / Render 擇一) ，提供可存取的 https:// URL 。
- 採 AI-only workflow 開發，README 須記錄使用工具與 Agent 工作流程 (規劃 / 開發 / Code Review / 測試) 。
- 保留完整、語意化的 Commit History (禁止單一 “final version” commit) 。
- 提供 MCP Client 驗證腳本與 Server Log (可看到 Tool Invoked、Query、Retrieved Chunks) 。

二、使用技術的選擇

技術選型以「保留文件結構與語意」「中英混雜檢索品質」「部署與維運成本」三個面向為主要考量。

面向	選用技術	選擇理由
語言	Python 3.11	文件解析與 ML / Embedding 生態最完整；Docling、sentence-transformers、Chroma 均為 Python。
Web API	FastAPI	原生 async / ASGI，可將 MCP 的 ASGI app 直接 mount 於 /mcp，單一服務同時提供 REST 與 MCP。
解析	Docling	輸出帶結構的文件 (標題 / 表格 / 圖片 + 頁碼 / 投影片 provenance)，優於純 OCR 只得到扁平文字。
圖片文字	RapidOCR	Docling 的 PPTX/DOCX 管線不對嵌入圖片做 OCR；補上後圖表型投影片的文字才不會遺失。
稠密檢索	BGE-M3 (sentence-transformers)	多語模型 (1024 維)，對中英混雜語料表現佳；L2 正規化後 cosine 等同內積。
稀疏檢索	BM25 (rank-bm25)	精確詞與罕見 token (專有名詞、CLAUDE.md 這類識別字) 常被 embedding 漏掉，BM25

		可補。
融合	Reciprocal Rank Fusion (RRF)	以名次而非分數融合，免去 dense/sparse 分數尺度不一致與權重調參問題。
向量庫	Chroma (embedded, 持久化)	內嵌於同一進程、不需獨立服務即可持久化，最貼合單服務部署與作業規模。
MCP 框架	FastMCP	以 streamable HTTP transport 提供 Remote MCP；工具 / 資源以裝飾器註冊，內建 in-memory client 便於測試。
部署	Docker + Zeabur	Dockerfile 一鍵建置；Zeabur 自動偵測 Dockerfile，提供 Volume 與公開網域。
測試	Pytest	單元、協定、整合測試一致；in-memory MCP client 可做 hermetic 端對端測試。

2.1 關鍵取舍說明

- 為何 Docling 而非簡單 OCR：Docling 會還原頁碼 / 投影片來源與表格列欄結構，讓後續切塊能沿語意邊界進行；純 PDF OCR 只會得到失去結構的文字流。
- 為何 Hybrid 而非單一檢索：純 dense 對精確詞與罕見識別字召回不穩，純 SQLite FTS 又缺乏語意泛化；Hybrid 兩者互補。題目亦明確要求不得僅用 FTS。
- 為何 RRF 融合：dense 的 cosine 相似度與 BM25 分數尺度完全不同，直接加權需人工調參且不穩定；RRF 只看名次，穩健且免調參。
- 為何 embedded Chroma：避免額外維運一個向量資料庫服務；embedded 模式資料落於 Volume，重啟 / 重佈後索引仍在。

三、系統架構設計

3.1 資料流 (Pipeline)





3.2 模組分層 (app/)

模組	職責
app/main.py	FastAPI 應用工廠；掛載 REST 路由與 /mcp（MCP ASGI app），共用其 lifespan。
app/config.py	以 pydantic-settings 集中管理設定（模型、切塊大小、檢索參數等），可由環境變數覆寫。
app/models.py	跨模組共用的 Pydantic 模型（Document / ParsedElement / Chunk / SearchResult...）。
app/services/parser.py	Docling 解析 + RapidOCR；輸出 ParsedDocument。
app/services/cleaner.py	清洗管線：正規化空白、去重複頁首頁尾、去頁碼行與純符號段落。
app/services/chunker.py	具中繼資料的切塊；沿語意邊界、僅在元素之間切分。
app/services/embedder.py	BGE 稠密向量（延遲載入、L2 正規化）。
app/services/vector_store.py	Chroma 持久化索引：upsert / 稠密 search / 文件彙整。
app/services/retriever.py	Hybrid 檢索：稠密 + BM25，以 RRF 融合；中英混合 tokenizer。
app/services/pipeline.py	串起「解析→清洗→切塊→索引」的共用流程，供上傳端點與 CLI 共用。
app/mcp_server.py	FastMCP：註冊 4 個 tools 與 2 個 resources，並記錄工具呼叫 log。
app/api/documents.py	REST 端點：上傳（上傳即自動索引）與文件目錄

	查詢。
app/landing.py	首頁：動態列出目前註冊的 MCP tools / resources。

3.3 清洗規則 (保留結構的前提下去雜訊)

- 正規化空白字元 (表格保留其 Markdown 列結構) 。
- 移除在整份文件重複出現達門檻 (預設 ≥2 次) 的頁首 / 頁尾樣板文字 。
- 移除「只有頁碼」的行 (如 12、1 / 10、Page 3、第 5 頁) 與「只有符號、無實質字元」的段落 。
- 標題、表格、圖片一律保留；不對標題套用最小字數規則，避免誤刪短標題 。

3.4 切塊策略 (禁止固定字數切割)

- 每個表格、每張圖各自成為獨立 chunk (chunk_type = table / figure) 。
- 連續內文歸入「最近一個標題」開啟的 section；頁碼 / 投影片變動時必定另起新 chunk 。
- 僅當 section 超過軟上限 (chunk_max_chars，預設 1200) 時才切分，且只在「完整元素之間」切，不會切斷句子或表格列；延續的 chunk 沿用相同 section_title 。

3.5 混合檢索 (Hybrid Retrieval)

稠密召回處理語意相似 / 改寫；BM25 處理精確詞與罕見 token。兩份排名以 RRF 融合：

$$RRF(d) = \sum 1 / (k + rank_i(d)) \quad (k = 60, rank \text{ 為該 } id \text{ 在第 } i \text{ 份排名清單中的名次})$$

- Tokenizer 同時保留拉丁單字與個別 CJK 漢字，使中英混雜內容兩種檢索都命中 。
- BM25 語料以 Chroma collection 為唯一事實來源、延遲重建；新文件索引後呼叫 refresh() 即刻生效 。

3.6 上傳即索引 (免重啟)

POST /documents 會在同一進程內直接跑完整 pipeline (解析→清洗→切塊→embedding→Chroma 索引) 再刷新 BM25。因為與 MCP 工具共用同一組 get_vector_store() / get_retriever() 單例，文件上傳後 MCP 搜尋立即可見，不需重啟 Server。耗時的同步工作 (Docling / OCR / embedding) 以 thread pool 執行，避免阻塞 async event loop。

3.7 MCP 介面

類型	名稱	用途
Tool	search_documents(query, top_k)	混合檢索最相關的 chunk (回傳含 metadata 與分數) 。
Tool	list_documents()	列出已索引文件目錄。
Tool	get_document(document_id)	文件層級中繼資料 (chunk 數、頁碼 / 投影片範圍) 。
Tool	get_chunk(chunk_id)	單一 chunk 完整內容與中繼資料。
Resource	documents://all	所有已索引文件目錄。
Resource	documents://{document_id}	指定文件及其所有 chunk 。

四、開發過程

4.1 AI-only Workflow

全程以 Claude Code 進行 AI-only 開發。每個開發步驟遵循固定節奏：規劃 (輸出本步驟目標、原因、涉及檔案、測試方式、commit message) → 開發 → Code Review → 測試，並且一個步驟對應一個語意化 commit，嚴禁一次產生整個系統。前處理能力另封裝為 Claude Skill (即作業二)，本身也是 AI workflow 的一環。

4.2 分階段 Commit History (節錄)

步驟	Commit Message
Step 1 專案初始化	chore: initialize project
Step 2 文件上傳 + 目錄	feat: add document upload + catalogue
Step 3 Docling 解析	feat: add docling parser
Step 4 清洗管線	feat: add content cleaning pipeline
Step 5 具中繼資料切塊	feat: add metadata aware chunking
Step 6 Chroma 索引	feat: add chroma indexing
Step 7 混合檢索	feat: add hybrid retrieval
Step 8 MCP 工具	feat: add mcp tools
Step 9 MCP 資源	feat: add mcp resources
Step 10 整合測試	test: add mcp integration test

加值：上傳即索引	feat: auto-index documents on upload
加值：首頁	feat: add landing page with dynamic MCP tool listing
加值：部署 / Skill 封裝	chore: prepare for zeabur deployment ; feat: package ... claude skill

五、測試驗證手法

以 Pytest 建立四個層級的測試，全專案共 15 個測試檔、約 77 個測試案例。

層級	測試檔 (節錄)	驗證重點
單元測試	test_cleaner / test_chunker / test_parser(_pptx)	清洗規則、語意邊界切塊、metadata schema 完整性、頁碼 / 投影片來源。
索引 / 檢索	test_index / test_retriever	Chroma upsert 與查詢；RRF 融合與中英混合 tokenizer 行為。
MCP (直呼)	test_mcp_tools / test_mcp_resources	四個工具與兩個資源函式的輸出結構與錯誤處理。
MCP (協定)	test_mcp_integration	以 FastMCP in-memory Client 走真實 JSON-RPC：discovery、輸入 schema、結構化輸出、resource URI。
REST	test_upload / test_health	上傳型別驗證、大小上限、上傳即索引、健康檢查。

5.1 端對端 MCP 驗證 (Client → Server → Tool → Result)

除自動化測試外，另提供可對「執行中 Server」操作的 client demo，走真實 MCP 協定：

```
# 終端機 1：啟動 Server
kb_mcp_env\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000

# 終端機 2：連線 Remote MCP Client 執行範例查詢
kb_mcp_env\Scripts\python.exe tests\mcp_client_demo.py "yield improvement plan"
```

此時 Server console 會輸出可驗證的工具呼叫紀錄：

```
INFO:app.mcp_server:MCP tool invoked: search_documents | query='...' top_k=3
INFO:app.mcp_server:search_documents retrieved 3 chunk(s): [...]
```

5.2 範例查詢 (README 提供)

```
What is the yield improvement plan?
Show me the KPI table from Q4 report.
Summarize slide 5.
```

六、上線網址與使用方式

6.1 公開部署網址

部署平台：Zeabur (自動偵測 Dockerfile 建置，掛載 Volume 於 /data 以持久化索引、上傳檔與模型快取) 。

公開網址：<https://enterprise-knowledge-mcp-server.zeabur.app>

端點	說明
https://enterprise-knowledge-mcp-server.zeabur.app/	首頁：專案介紹與動態 MCP 工具 / 資源清單
https://enterprise-knowledge-mcp-server.zeabur.app/health	健康檢查 (liveness probe)
https://enterprise-knowledge-mcp-server.zeabur.app/mcp	Remote MCP 端點 (供 Claude Desktop / Claude Code 連線)
https://enterprise-knowledge-mcp-server.zeabur.app/docs	REST API 文件 (Swagger UI)

6.2 連接 Claude Desktop / Claude Code

將 MCP client 指向部署的 /mcp 端點即可使用四項工具與兩項資源。

6.3 新增文件 (上傳即索引、免重啟)

```
curl.exe -X POST https://<部署網址>.zeabur.app/documents -F
"file=@C:\path\to\report.pdf"
# -> 201 {"document_id": "...", "status": "indexed", "num_chunks": 42}
```

6.4 本機執行 / Docker

```
# 本機
kb_mcp_env\Scripts\python.exe -m pip install -e "[dev]"
```

```
kb_mcp_env\Scripts\python.exe -m pytest -q  
kb_mcp_env\Scripts\python.exe -m uvicorn app.main:app --reload
```

```
# Docker  
docker compose up --build # http://localhost:8000
```

作業二 document-preprocessing Claude Skill

一、系統需求 (背景與動機)

作業一的前處理管線 (Docling 解析 → 清洗 → 具中繼資料切塊) 是整套系統中最具重用價值的一段邏輯。本作業將它抽出、封裝為一個可重用的 Claude Skill，使 Claude Code 或任何 Agent 能在不啟動 HTTP / MCP Server、也不觸碰向量資料庫的前提下，對單一 DOCX/PDF/PPTX 檔即時做前處理，並產出「index-ready 的 JSON」與「人類可讀的 Markdown」。

1.1 需求要點

- 單一檔案輸入、清楚的輸入 / 輸出契約，可被 Agent 自動判斷何時該使用。
- 重用作業一的生產程式碼，不得另起爐灶重複實作 (維持單一事實來源)。
- 安全邊界明確：唯讀輸入、限定副檔名、大小上限、輸出目錄限制、不寫 DB、不對外連網。
- 輸出的每個 chunk 需符合與作業一相同的 metadata schema。

二、使用技術的選擇

面向	選用作法	理由
技能格式	Claude Skill (SKILL.md + scripts/)	以 SKILL.md 的 name / description frontmatter 讓 Agent 自動判斷觸發時機；Claude Code 可自動探索。
實作方式	薄封裝 (thin wrapper)	直接呼叫 app/services 的 parser / cleaner / chunker，重用生產邏輯，確保輸出與知識庫完全一致。
介面	argparse CLI	以旗標控制 stage / format / 輸出目錄等；純命令列、易被 Agent 與 CI 呼叫。
I/O 分流	stdout / stderr 分離、UTF-8	進度 log 走 stderr、結構化 JSON 走 stdout，兩者不混；UTF-8 避免 Windows cp950

		對中文崩潰。
相依	沿用專案 <code>venv</code>	共用 Docling / RapidOCR，無需額外安裝。

三、系統架構設計

3.1 目錄結構

```
.claude/skills/document-preprocessing/
├─ SKILL.md                # 技能說明+觸發描述 (frontmatter)
├─ scripts/preprocess.py    # 統一前處理 CLI (薄封裝)
├─ reference/OUTPUT_SCHEMA.md # 輸出 JSON/metadata schema
└─ examples/RUN_LOG.md      # 真實執行紀錄與產物 (驗證證據)
```

3.2 處理流程

```
輸入檔 → Docling parse → clean (去雜訊) → metadata-aware chunk → JSON +
Markdown
```

`preprocess.py` 會向上尋找專案根目錄 (以 `app/services/parser.py` 為錨點)，將其加入 `sys.path` 後直接匯入生產模組，因此無論目錄如何搬移都能運作，且輸出邏輯與作業 100% 相同。

3.3 輸入 / 輸出契約

選項	預設	意義
<code>--stage {parse,clean,chunk,all}</code>	<code>all</code>	要執行的管線階段。
<code>--format {json,md,both}</code>	<code>both</code>	輸出哪種產物。
<code>--out DIR</code>	<code>./skill_output</code>	輸出目錄 (不存在則建立)。
<code>--doc-id ID</code>	檔名 slug	document id / chunk-id 前綴。
<code>--max-chars N</code>	1200	切塊軟上限 (僅在元素邊界切)。
<code>--max-mb N</code>	50	超過即拒絕的輸入大小上限。
<code>--no-ocr</code>	<code>off</code>	關閉圖片 OCR (較快)。
<code>--stdout / --quiet</code>	<code>off</code>	同時輸出最終 JSON 到 <code>stdout</code> / 抑制進度 <code>log</code> 。

輸出檔 (依輸入檔名 slug 命名) : parse→<stem>.parsed.*、clean→<stem>.cleaned.*、chunk→<stem>.chunks.* ; all 產出全部中間產物。

退出碼 : 0 成功 / 1 輸入或 IO 錯誤 / 2 用法或驗證錯誤。

3.4 安全邊界

- 輸入檔以唯讀開啟，永不修改。
- 僅接受 .docx / .pdf / .pptx ; 超過大小上限即拒絕。
- 所有寫入都限制在 --out 目錄內，拒絕任何路徑逃逸 (path escape) 。
- 不寫入任何資料庫、不對外連網 (模型檔僅由 Docling / RapidOCR 於本機快取下載一次) 。
- 發生錯誤只輸出一行 error: ... 與非零退出碼，不噴 traceback 。

四、開發過程

在作業一的十個步驟完成、pipeline 穩定後，才將前處理三段 (parser/cleaner/chunker) 以薄封裝方式抽出為 Skill，對應 commit : feat: package preprocessing pipeline as document-preprocessing claude skill。設計原則為 DRY——Skill 不複製任何解析 / 清洗 / 切塊邏輯，只負責 CLI 參數、驗證、輸出格式與安全邊界，確保它與線上知識庫產出的 chunk 完全一致。

五、測試驗證手法

- 自動化測試 tests/test_skill_preprocess.py 共 8 個案例，端對端驗證 CLI : 全階段產物、清洗去樣板與頁碼、chunk metadata schema 完整、自訂 doc-id 前綴、拒絕不支援副檔名 / 不存在檔 / 超大檔、json-only 略過 markdown 。
- examples/RUN_LOG.md 保存對一份真實 15MB 企業 PDF 的實際執行紀錄與產物：解析 17 個元素、切出 13 個 chunk，與該文件在知識庫中的索引 chunk 數一致——證明 Skill 與生產管線輸出完全相同。

```
$ kb_mcp_env\Scripts\python.exe -m pytest tests/test_skill_preprocess.py -v
... 8 passed in 11.12s
```

六、交付與使用方式

此 Skill 隨程式庫一併提交，Claude Code 會自動探索（不需另行部署 URL）。

位置：`.claude/skills/document-preprocessing/`

GitHub：<https://github.com/j84077200345-dotcom/Enterprise-Knowledge-MCP-Server>

6.1 呼叫範例

```
# 完整管線（parse+clean+chunk），輸出 JSON+Markdown 至 ./skill_output
kb_mcp_env\Scripts\python.exe \
.claude/skills/document-preprocessing/scripts/preprocess.py \
"path\to\report.pdf" --stage all --out skill_output

# 只解析投影片、看原始結構與投影片編號
preprocess.py "deck.pptx" --stage parse

# 為索引切塊、只要 JSON、關閉 OCR 加速
preprocess.py "doc.docx" --stage chunk --format json --no-ocr
```

結語

作業一交付一套符合題目所有硬性要求的 Production-Ready 企業知識庫：Docling 解析、結構保留的清洗、禁固定字數的語意切塊、Dense+BM25 混合檢索、FastMCP Remote MCP Server、公開部署、完整語意化 commit history、以及可驗證的 MCP client 與 server log。作業二則將其中最具有重用價值的前處理能力封裝為 Claude Skill，以薄封裝重用同一份生產程式碼，示範「同一套邏輯、兩種交付形態」的工程實踐。兩份作業皆以 AI-only workflow 完成。